

PocketSmartAI—YourSmartBudget& Recommendation Assistant

ProjectDocumentation

AnAI-poweredpersonalfinancecompanionthattracksspending,buildssmart budgets, and delivers personalized money-saving recommendations.

DocumentType	ProjectDocumentation
ApplicationCategory	AI/FinTech—PersonalBudgeting
Platform	WebApplication(React.jsFrontend)
Version	1.0

TEAMMEMBERS	EMAILID
M.BALA SURYA	Suryam9307@gmail.com
P.BARATH	barath89250@gmail.com
M.BASKAR	baskarscas@gmail.com
B.BHARATHIDHASAN	bharathibharathi20622@gmail.com
S.CHANDRU	chandru123456rs@gmail.com

G.MOHAMED RIFA	@gmail.com
	lw7458372@gmail.com

PROJECT OVERVIEW

Pocket Smart AI is an intelligent web application designed to help users take control of their personal finances effortlessly. It combines automated expense tracking, AI-driven budget planning, and personalized recommendations to help users save more, spend wisely, and reach their financial goals. The platform is built for everyone — from students managing pocket money to professionals planning long-term savings.

Purpose

PocketSmart AI aims to make personal finance management simple, insightful, and proactive rather than reactive. The main goals are:

- **Smart Budgeting**—AI-generated monthly budgets based on income, spending habits, and goals.
- **Personalized Recommendations**—Actionable suggestions to cut costs and optimize savings.
- **Effortless Expense Tracking**—Automatic categorization of transactions and spending patterns.
- **Modern Tech Stack**—Built using React.js, Node.js, and a machine-learning recommendation engine.

Features

- **AI Budget Engine**—Generates a personalized monthly budget from income and expense data.
- **Smart Recommendations**—Suggests where to cut spending and how to reach savings goals faster.
- **Expense Tracker**—Logs and categorizes transactions automatically (food, bills, travel, etc.).
- **Goal Planner**—Lets users set savings goals and tracks progress in real time.
- **Interactive Dashboards**—Visual charts summarizing spending trends and budget health.
- **Alerts & Insights**—Notifies users when nearing budget limits or of unusual spending.

ARCHITECTURE

Component Structure

The application is divided into three main sections:

- **Pages**—Full-page views (Dashboard, Budget Planner, Recommendations, Reports).
- **Components**—Reusable UI elements (Navbar, Expense Cards, Charts, Goal Tracker).
- **Styles**—CSS and styling files for consistent theming.

State Management

- **Global State**—Managed using React Context API for user profile, budget data, and preferences.
- **Local State**—Controlled via React's `useState` for component-level interactions.

Routing: Implemented using React Router to enable seamless navigation between the Dashboard, Budget Planner, Recommendations, and Reports pages.

SETUP INSTRUCTIONS

Prerequisites

- Node.js & npm – Install from the official Node.js website.
- React.js – Set up a new project using create-react-app.

Installation Steps

- Clone the repository
- Navigate into the project directory
- Install dependencies – `run npm install`
- Set up environment variables by creating a `.env` file with the required API keys (e.g., recommendation engine key, currency API key).
- Start the development server – `run npm start`

Example commands:

```
git clone https://github.com/your-org/pocket-smart-ai cd
pocket-smart-ai
npm install
npm start
```

Once started, open `http://localhost:3000` in your web browser to access the application.

FOLDER STRUCTURE

The project is structured into different directories for better organization and maintainability.

```
pocket-smart-ai/
src/
  components/ #Navbar, Cards, Charts, Modals
  pages/ #Dashboard, Budget, Recommendations, Reports
  context/ # BudgetContext.js, UserContext.js
  services/ # API calls to the recommendation engine
  styles/ # CSS / SCSS files
  assets/ #Icons and images public/
package.json
```

Client

The client folder (inside `src`) contains the core files of the frontend application. It consists of:

- Components – Houses reusable UI componentssuch as budget cards, charts, and the navigation bar.

- Pages—Contains full-page components, including the dashboard, budget planner, and reports page.
 - Styles—Includes all CSS/SCSS files used to style the application.
 - Assets—Stores icons, illustrations, and other static files used throughout the app.
-

RUNNING THE APPLICATION

To start the frontend server, run:

```
npm start
```

Then open `http://localhost:3000` in your browser to access the dashboard.

COMPONENT DOCUMENTATION

Key Components

Below are the major components of the PocketSmartAI application, along with their purpose and props:

NavbarComponent (Navbar.js)

Purpose:

- Provides site-wide navigation.
- Contains links to Dashboard, Budget, Recommendations, and Reports.

Props:

- `logo(string)`: Path to the logo image.
- `menuItems(array)`: List of navigation menu items.

DashboardSummaryComponent (DashboardSummary.js)

Purpose:

- Displays an overview of income, expenses, and remaining budget.
- Shows quick-glance charts for the current month.

Props:

- `income(number)`: Total monthly income.
- `expenses(number)`: Total monthly expenses.
- `budget(object)`: Current budget allocation by category.

ExpenseCardComponent (ExpenseCard.js)

Purpose:

- Displays a single transaction with category, amount, and date.

- Allows user to edit or delete an entry.

Props:

- `expense(object)`: Contains `id`, `title`, `amount`, `category`, and `date`.

RecommendationCardComponent (RecommendationCard.js)

Purpose:

- Displays an AI-generated saving tip or budget adjustment suggestion.
- Lets users accept or dismiss a recommendation.

Props:

- `recommendation(object)`: Contains `id`, `message`, `category`, and `potential savings`.

BudgetPlannerComponent (BudgetPlanner.js)

Purpose:

- Allows user to set or adjust category-wise budget limits.
- Visualizes allocation versus actual spending.

Props:

- `categories(array)`: List of budget categories and limits.

GoalTrackerComponent (GoalTracker.js)

Purpose:

- Displays progress toward a user-defined savings goal.

Props:

- `goal(object)`: Contains `name`, `targetAmount`, and `currentAmount`.

Reusable Components

Search/ FilterBarComponent (FilterBar.js)

Purpose:

- Allows user to filter transactions by category, date, or amount.

Props:

- `onFilter(function)`: Callback function for handling filter changes.

ButtonComponent (Button.js)

Purpose:

- A reusable button component with customizable styles.

Props:

- `text(string):` Button label.
- `onClick(function):` Click event handler.
- `variant(string):` Defines button style (e.g., `primary`, `secondary`).

LoadingSpinnerComponent (Loading.js)**Purpose:**

- Displays a loading animation while fetching data or generating recommendations.

Props:

- `size(string):` Size of the spinner (`small`, `medium`, `large`).
-

STATE MANAGEMENT

State management in the PocketSmart AI application ensures efficient data handling across different components. The project incorporates both global state for shared data and local state for component-specific interactions.

Global State

The application uses the React Context API to manage global state, allowing components to access shared data without prop drilling.

- `BudgetDataStorage`—Stores income, expenses, and category budgets.
- `RecommendationState`—Manages AI-generated suggestions and their status (accepted/dismissed).
- `UserPreferences`—Keeps track of savings goals and notification settings.

How global state flows across the application:

- `TheBudgetContext.js` file initializes and manages the global state.
- `TheBudgetProvider` component wraps the application to provide access to the global state.
- Components like `Dashboard.js` and `Recommendations.js` consume global state using `useContext`.
- Any update to the global state (e.g., a new transaction) automatically reflects across all dependent components.

Local State

Local state is managed using the `useState` hook within individual components for temporary UI interactions that don't need to persist across the application.

- `FilterBar`—Stores and updates the user's active filter selection.
- `BudgetPlanner`—Keeps track of unsaved changes to category limits.

- UIComponents—Controls modals,dropdownmenus,andloading states.
-

USERINTERFACE

ThePocketSmartAIapplicationfeaturesaclean,data-forwardinterfacedesignedtomakefinancialinsights easy to understand at a glance.

Dashboard(Overview&Quick Insights)

Features:

- Asummarybannershowingincome,expenses, andremaining budget.
- Visualchartsofspendingbycategory.
- Aquick-accesspanelforthe latestAIrecommendations.

BudgetPlannerPage

Features:

- Category-wisebudgetallocationwithslidersandinputfields.
- Progressbarscomparingplannedversusactualspending.
- One-clickoptiontoapplyAI-suggestedbudgetadjustments.

RecommendationsPage

Features:

- Listspersonalizedsavingtips generatedbytheAI engine.
- Eachcardshowspotentialmonthlysavingsandashortexplanation.
- Userscanaccept,dismiss,orsavearecommendation forlater.

Reports&AnalyticsPage

Features:

- Monthlyandyearlyspendingtrendsinchartform.
- Breakdownofexpensesbycategoryand merchant.
- Exportablesummariesforpersonalrecord-keeping.

ResponsiveUI&MobileView

Features:

- Theapplicationis fullyresponsiveacrossdifferentcreensizes.
 - Mobile-friendlynavigationmenuandswipeabletransaction cards.
-

STYLING

PocketSmartAI is designed with a modern, calm, and trustworthy aesthetic appropriate for a finance application, using industry-standard styling techniques.

CSS Frameworks & Libraries Used

- Tailwind CSS—A utility-first framework used for fast, consistent, and responsive styling.
- Recharts/Chart.js—Used to render budget and spending visualizations.
- React Icons—Used for navigation, category, and status icons throughout the UI.

Theming & Custom Design

- Consistent Color Scheme—A calm green and neutral palette conveys trust and financial clarity.
- Primary Color—Used for buttons, highlights, and key call-to-action elements.
- Secondary Color—Used for backgrounds and supporting UI elements.
- Dark Mode & Light Mode—Users can toggle between modes to reduce eyestrain.

Animations & Transitions

- Smooth chart animations when budget data updates.
 - Fade-in transitions for newly generated AI recommendations.
 - Lazy loading for report data to keep the app lightweight and fast.
-

TESTING

Ensuring the reliability and accuracy of financial calculations and recommendations is critical. The testing approach includes multiple strategies.

Testing Strategy

Unit Testing:

- Used Jest and React Testing Library to test individual components.
- Verified budget calculations and category totals render correctly.

Integration Testing:

- Verified that the recommendation engine returns and displays suggestions correctly.
- Ensured that updating a transaction refreshes the dashboard and reports in real time.

End-to-End Testing:

- Used Cypress to simulate real user workflows, including adding an expense, setting a budget, and reviewing a recommendation.

Code Coverage & Goals

- Achieve 80%+ test coverage, focusing on budget calculations, recommendation logic, and UI rendering.
 - Implement CI/CD pipeline to run tests automatically before deployment.
-

KNOWN ISSUES

Despite extensive testing, the PocketSmart AI application may have some known issues that could affect user experience.

Slow Recommendation Generation

Issue:

- Recommendation data occasionally takes long to load when the AI engine is processing a large transaction history.

Possible Cause:

- High volume of transactions or network latency.

Potential Solution:

- Implement caching for recent recommendations and show a loading indicator while processing.

Category Misclassification

Issue:

- Some transactions are placed in the wrong spending category.

Possible Cause:

- The auto-categorization logic does not always match uncommon merchant names.

Potential Solution:

- Allow users to manually recategorize transactions, feeding corrections back into the model.

UI Overlapping on Mobile Screens

Issue:

- Some UI components (e.g., charts, navbar) overlap on smaller screens.

Possible Cause:

- Missing proper media queries for responsive design.

Potential Solution:

- Adjust CSS breakpoints and use flexbox/grid layouts for dynamic positioning.

NoFeedbackonFailedAPICalls

Issue:

- When therecommendationAPIfails,usersdon'treceiveanerror message.

PossibleCause:

- NoerrorhandlingimplementedinAPI calls.

PotentialSolution:

- Implementtry-catch blocksanddisplayfallbackUIwitha retry option.

Theseissueswillbeaddressedinfutureupdatestoenhancetheperformance,usability,andreliabilityofthe Pocket Smart AI application.

FUTUREENHANCEMENTS

PocketSmartAIsdesignedtoprovideaseamlessexperienceforbudgetingandfinancialplanning.Below are potential future enhancements.

UserAuthentication&Multi-DeviceSync

Enhancement:

- Implementsecuresign-up/loginandsyncbudgetdata acrossdevices.

Benefit:

- Enablesuserstoaccesstheirfinancialdataanywhere,securely.

BankAccountIntegration

Enhancement:

- Connectdirectlytobankaccountsforautomatictransactionimport.

Benefit:

- Removessthe needfor manualentryandimprovesdata accuracy.

PredictiveSpendingAlerts

Enhancement:

- UseAItoforecastupcomingexpensesandwarnusers beforeoverspending.

Benefit:

- Helpsusersstayaheadof their budgetinsteadof reacting afterthefact.

BillReminder&Auto-Categorization

Enhancement:

- Automatically detect recurring bills and remind users before due dates.

Benefit:

- Reduces missed payments and late fees.

Investment & Savings Suggestions

Enhancement:

- Recommends savings or investment options based on surplus income.

Benefit:

- Helps users grow wealth beyond basic budgeting.

Voice-Activated Finance Assistant

Enhancement:

- Integrate voice commands for quick expense logging and balance checks.

Benefit:

- Makes daily tracking faster and more convenient.